ISPilot Teams Integration with Microsoft 365 Agents SDK

Date: 2026-09-01
Status: Planning
Target Deployment: Cloud Run service `teams-ispilot-bridge`

Overview

ISPilot will be integrated into Microsoft Teams using the **Microsoft 365 Agents SDK** (Python). This approach allows Teams users to interact with the ispilot-api without reimplementing business logic.

Architecture Flow:

```
Teams User
    ↓
Microsoft Teams Client sends message to Azure Bot Service
    ↓
Azure Bot Service routes Activity JSON + JWT token to Bridge
    ↓
Teams Bridge (Cloud Run service)
├─ Validates JWT token from Azure
├─ Extracts message context
├─ Generates identity token for sa-tot-osa
    ↓
ispilot-api (Private Cloud Run service)
    ↓
Vertex AI Reasoning Engine
    ↓
Response flows back through Bridge to Teams
```

Prerequisites

Azure / Microsoft 365 Resources

1. **Azure AD Tenant** with Teams admin access
2. **Teams Developer Portal** or **Azure Portal** access
3. Ability to create Bot registrations

Google Cloud Resources

- Project: `corp-stro-salesinventory-prod`
- Service Account: `sa-tot-osa@corp-stro-salesinventory-prod.iam.gserviceaccount.com` (already configured)

- ispilot-api: `https://ispilot-api-46y2f3tyja-uc.a.run.app` (already deployed)

Setup Steps

Phase 1: Register Bot in Azure

Who: Azure/Teams admin (likely José Arturo)

1. Go to **Azure Portal** → **App registrations** (or Teams Developer Portal)
2. Create new app registration:
   - Name: `ISPilot-Teams-Bridge`
   - Supported account types: Single tenant
3. Copy the following and save securely:
   - **Application (Client) ID** → `MICROSOFT_APP_ID`
   - **Directory (Tenant) ID** → `MICROSOFT_TENANT_ID`
4. Create client secret:
   - Go to **Certificates & secrets** → **New client secret**
   - Copy the value → `MICROSOFT_APP_PASSWORD`
5. Register OAuth redirect URI:
   - **Authentication** → **Redirect URIs** → Add:
     - `https://teams-ispilot-bridge-xxxxxx.run.app/auth/callback`
     (exact URL TBD after first Cloud Run deploy)

Phase 2: Create Teams Bridge Service

Who: Development team

1. **Clone or create** `teams_bot_bridge/` directory in root

2. **Install dependencies**:

```
cd teams_bot_bridge
python -m venv .venv
source .venv/bin/activate  # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

requirements.txt:

```
fastapi==0.104.1
uvicorn==0.24.0
microsoft-365-agents-sdk==0.1.0  # Verify version
google-auth==2.25.0
google-cloud-logging==3.8.0
python-dotenv==1.0.0
httpx==0.25.0
pydantic==2.5.0
```

3. **Create configuration file** `a365.config.json`:

```
{
  "appId": "<MICROSOFT_APP_ID>",
  "appPassword": "<MICROSOFT_APP_PASSWORD>",
  "messagingEndpoint": "https://teams-ispilot-bridge-
xxxxxx.run.app/api/messages",
  "ispilotApiEndpoint": "https://ispilot-api-46y2f3tyja-
uc.a.run.app/chat",
  "serviceAccountEmail": "sa-tot-osa@corp-stro-salesinventory-
prod.iam.gserviceaccount.com",
  "googleCloudProject": "corp-stro-salesinventory-prod"
}
```

4. **Create main application** `main.py`:

```
from fastapi import FastAPI, Request, HTTPException, Depends
from fastapi.responses import JSONResponse
from microsoft.agents.sdk import AgentApplication, AgentAuthConfiguration
from microsoft.agents.sdk.models import Activity, ActivityTypes
import httpx
import google.auth
from google.auth.transport.requests import Request as GoogleRequest
import json
import os
from dotenv import load_dotenv

load_dotenv()

app = FastAPI()

# Load config
with open("a365.config.json") as f:
    config = json.load(f)

# Initialize Agents SDK with JWT validation
auth_config = AgentAuthConfiguration(
    app_id=config["appId"],
    app_password=config["appPassword"]
)

agent = AgentApplication(auth_config=auth_config)

@agent.activity("message")
async def on_message(activity: Activity, context):
    """Handle incoming Teams messages"""
    user_message = activity.text
    user_id = activity.from_.id
```

```
    conversation_id = activity.conversation.id

    try:
        # Generate identity token for sa-tot-osa
        credentials, _ = google.auth.default(
            scopes=["https://www.googleapis.com/auth/cloud-platform"]
        )
        google_request = GoogleRequest()
        credentials.refresh(google_request)
        identity_token = credentials.token

        # Call ispilot-api
        async with httpx.AsyncClient() as client:
            response = await client.post(
                config["ispilotApiEndpoint"],
                headers={
                    "Authorization": f"Bearer {identity_token}",
                    "Content-Type": "application/json"
                },
                json={
                    "user_id": user_id,
                    "message": user_message,
                    "session_id": conversation_id
                },
                timeout=30.0
            )
            response.raise_for_status()
            api_response = response.json()

        # Extract answer and send back to Teams
        answer = api_response.get("answer", "No answer received")
        await context.send_activity(answer)

    except Exception as e:
        error_msg = f"Error processing message: {str(e)}"
        await context.send_activity(error_msg)

@app.post("/api/messages")
async def handle_messages(request: Request):
    """Endpoint for Azure Bot Service to send Teams activities"""
    body = await request.json()
    activity = Activity.deserialize(body)

    # Let Agents SDK handle JWT validation and routing
    return await agent.process_activity(activity)

@app.get("/health")
async def health_check():
    return {"status": "ok"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8080)
```

5. **Create Dockerfile**:

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
```

6. **Create** `.env.example`:

```
MICROSOFT_APP_ID=<your-app-id>
MICROSOFT_APP_PASSWORD=<your-app-password>
GOOGLE_CLOUD_PROJECT=corp-stro-salesinventory-prod
```

Phase 3: Deploy Teams Bridge to Cloud Run

Who: Development team with GCP permissions

```
cd teams_bot_bridge

# Build and push to Cloud Run
gcloud run deploy teams-ispilot-bridge \
  --source . \
  --region us-central1 \
  --project corp-stro-salesinventory-prod \
  --service-account sa-tot-osa@corp-stro-salesinventory-
prod.iam.gserviceaccount.com \
  --set-env-vars MICROSOFT_APP_ID=<APP_ID>,MICROSOFT_APP_PASSWORD=
<APP_PASSWORD> \
  --allow-unauthenticated \
  --memory 512Mi \
  --timeout 60
```

After deployment, note the Cloud Run URL and:

1. Update `a365.config.json` with `messagingEndpoint`
2. Redeploy if needed
3. Update Azure Bot registration with new messaging endpoint

Phase 4: Configure Azure Bot Service

Who: Azure/Teams admin

1. Go to **Bot Service** in Azure Portal
2. Set **Messaging endpoint**:

```
https://teams-ispilot-bridge-xxxxxx.run.app/api/messages
```

3. Test connection with **Test in Web Chat**
4. Create Teams app manifest

Phase 5: Teams Manifest & Publishing

Who: Teams admin

1. **Create manifest.json**:

```
{
  "$schema": "https://developer.microsoft.com/json-
schemas/teams/v1.16/MicrosoftTeams.schema.json",
  "manifestVersion": "1.16",
  "version": "1.0.0",
  "id": "<MICROSOFT_APP_ID>",
  "name": {
    "short": "ISPilot",
    "full": "ISPilot - Retail AI Assistant"
  },
  "description": {
    "short": "Natural language access to retail operational data",
    "full": "ISPilot provides Teams users with AI-powered insights into
store performance, inventory, and operational metrics"
  },
  "icons": {
    "outline": "outline_icon.png",
    "color": "color_icon.png"
  },
  "accentColor": "#004578",
  "bots": [
    {
      "botId": "<MICROSOFT_APP_ID>",
      "scopes": ["personal", "team"],
      "commandLists": [
        {
          "scopes": ["personal", "team"],
          "commands": [
            {
              "title": "Help",
              "description": "Show help menu"
```

```
            }
          ]
        }
      ],
      "supportsFiles": false,
      "isNotificationOnly": false
    }
  ],
  "permissions": ["identity", "messageTeamMembers"],
  "validDomains": ["teams-ispilot-bridge-xxxxxx.run.app"]
}
```

2. **Upload to Teams**:
    - Teams Admin Center → Manage apps → Upload custom app
    - Select manifest.json
    - Publish to organization/catalog

Testing Checklist

- ☐ Teams Bridge Cloud Run service is running
- ☐ Health check passes: `curl https://teams-ispilot-bridge-xxxxxx.run.app/health`
- ☐ Azure Bot Service test accepts messages
- ☐ Teams app can be added to Teams client
- ☐ Send test message: "How is Talca Colin performing?"
- ☐ Receive response from ispilot-api through Teams
- ☐ Session persistence works (multi-turn conversation)
- ☐ Error handling works (invalid questions, timeouts)

Authentication Flow Validation

Working production test (once Bridge is deployed):

```
# 1. Get Teams service account token
TOKEN=$(gcloud auth print-identity-token \
  --impersonate-service-account=sa-tot-osa@corp-stro-salesinventory-
prod.iam.gserviceaccount.com)

# 2. Simulate Teams Activity to Bridge
curl -X POST "https://teams-ispilot-bridge-xxxxxx.run.app/api/messages" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <MICROSOFT_BEARER_TOKEN>" \
  --data '{
    "type": "message",
    "from": {"id": "test-user", "name": "Test User"},
    "conversation": {"id": "conv-123"},
```

```
    "text": "How is Talca Colin performing?"
  }'
```

Known Limitations & Mitigations

Issue	Cause	Mitigation
Message timeout	ispilot-api takes >15s	Implement streaming responses or "typing" indicator
Large responses	Teams message size limit (~28KB)	Truncate or paginate responses
Session loss	Firestore regional issue	Use in-memory fallback (already in ispilot-api)
Token expiry	Identity tokens expire	Implement token refresh in Bridge

---

Troubleshooting

Bridge not receiving messages

- Check Azure Bot messaging endpoint is set correctly
- Verify Cloud Run service is public (`--allow-unauthenticated`)
- Test with curl using Bot Framework token (not Google token)

JWT validation fails

- Confirm `MICROSOFT_APP_PASSWORD` is correctly set
- Regenerate secret in Azure if unsure
- Check Bridge logs: `gcloud logging read "resource.type=cloud_run_revision AND resource.labels.service_name=teams-ispilot-bridge"`

ispilot-api returns 401

- Confirm sa-tot-osa has Workload Identity permissions on Cloud Run
- Regenerate identity token:

```
gcloud auth print-identity-token \
  --impersonate-service-account=sa-tot-osa@corp-stro-salesinventory-prod.iam.gserviceaccount.com
```

References

- Microsoft 365 Agents SDK (Python)
- Azure Bot Service Documentation

- Teams App Manifest Schema
- Google Identity Tokens